

NYCU Introduction to Machine Learning, Homework 3

[112550018], [吳欣澤]

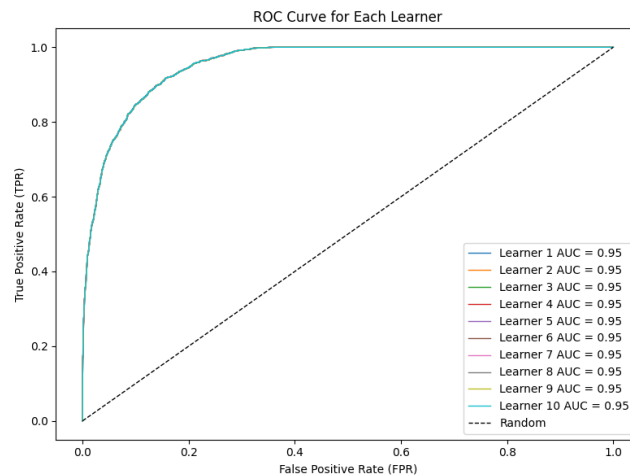
Part. 1, Coding (60%):

(20%) Adaboost

- (5%) Show your accuracy of the testing data ($n_estimators = 10$)

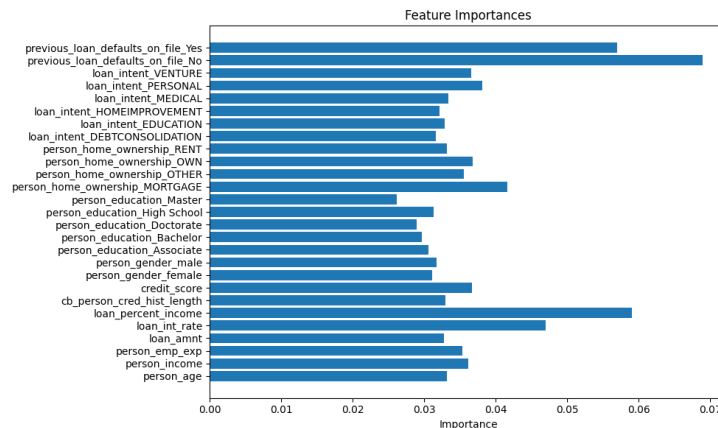
```
2025-11-18 22:00:11.381 | INFO | _main_:main:61 - AdaBoost - Accuracy: 0.8996
2025-11-18 22:00:14.730 | INFO | _main_:main:81 - Bagging - Accuracy: 0.8993
2025-11-18 22:01:06.495 | INFO | _main_:main:99 - DecisionTree - Accuracy: 0.9152
```

- (5%) Plot the AUC curves of each weak classifier.



- (10%)

- Plot the feature importance of the AdaBoost method.



- Paste the snapshot of your implementation of the feature importance estimation.

```
def compute_feature_importance(self) -> t.Sequence[float]:
    """
    TODO: Implement the feature importance calculation
    """
    importance = np.zeros(self.learners[0].fc1.in_features)
    for alpha, learner in zip(self.alphas, self.learners):
        importance += alpha * np.abs(learner.fc1.weight.detach().numpy()).sum(axis=0)
    return importance / np.sum(importance)
```

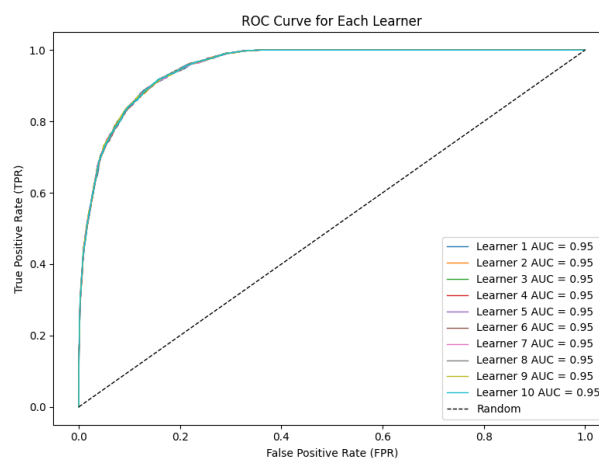
- c. Explain how you compute the feature importance for the Adaboost method shortly (within 100 words as possible)
- AdaBoost weights each learner's feature importance by its alpha value, which reflects its accuracy. More accurate learners have a greater influence on the final feature importance.

(20%) Bagging

4. (5%) Show the accuracy of the test data using 10 estimators. (`n_estimators=10`)

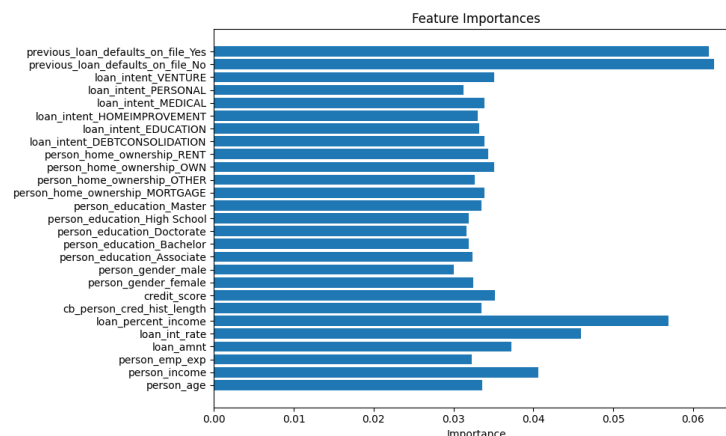
```
2025-11-18 22:00:11.381 | INFO | _main_:main:61 - AdaBoost - Accuracy: 0.8996
2025-11-18 22:00:14.730 | INFO | _main_:main:81 - Bagging - Accuracy: 0.8993
2025-11-18 22:01:06.495 | INFO | _main_:main:99 - DecisionTree - Accuracy: 0.9152
```

5. (5%) Plot the AUC curves of each weak classifier.



6. (10%)

- a. Plot the feature importance of the Bagging method.



- b. Paste the snapshot of your implementation of the feature importance estimation.

```
def compute_feature_importance(self) -> t.Sequence[float]:
    """
    TODO: Implement the feature importance calculation
    """
    importance = np.zeros(self.learners[0].fc1.in_features)
    for learner in self.learners:
        importance += np.abs(learner.fc1.weight.detach().numpy()).sum(axis=0)
    return importance / np.sum(importance)
```

- c. Explain how you compute the feature importance for the Bagging method shortly (within 100 words as possible)
- Bagging computes feature importance by summing the absolute weights of the first layer (fc1) from each weak classifier, then normalizing. Each learner contributes equally, regardless of accuracy.

(15%) Decision Tree

7. (5%) Compute the Gini index and the entropy of the array [0, 1, 0, 1, 0, 0, 0, 1, 0, 1, 1].

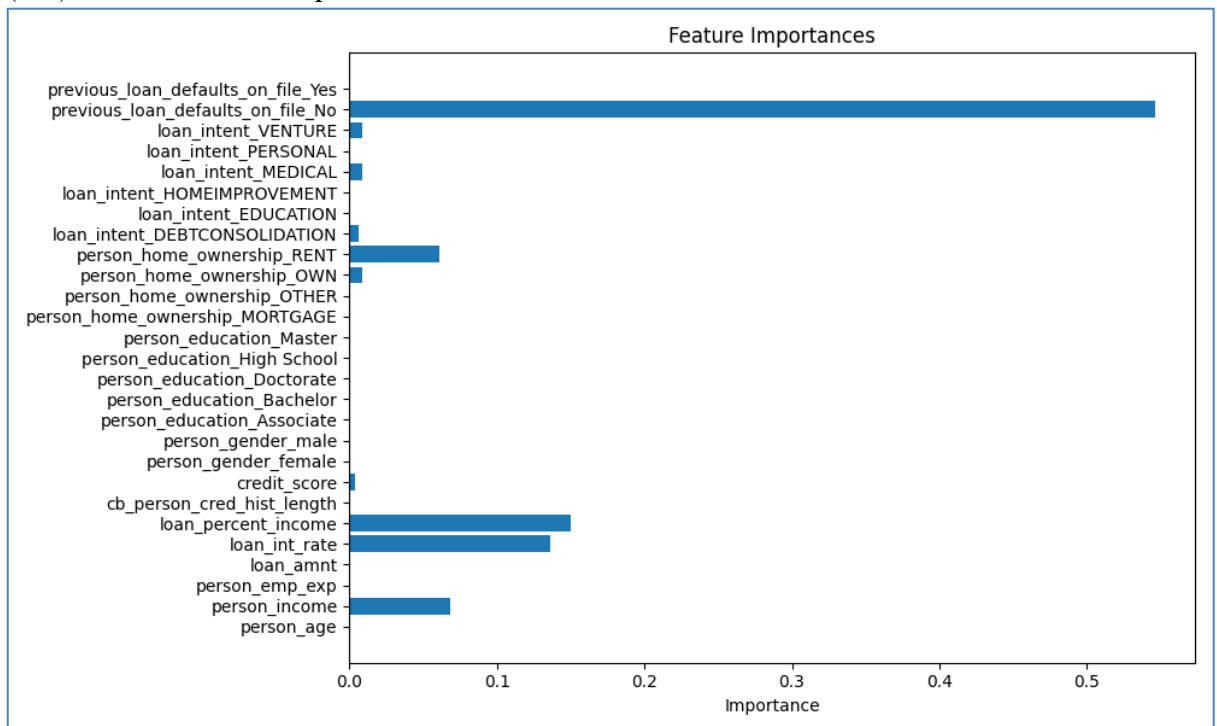
$$Gini = 1 - (p_0^2 + p_1^2) \approx 0.496$$

$$Entropy = -(p_0 \cdot \log_2(p_0) + p_1 \cdot \log_2(p_1)) \approx 0.994$$

8. (5%) Show your accuracy of the testing data with a max-depth = 7

```
2025-11-18 22:00:11.381 | INFO | __main__:main:61 - AdaBoost - Accuracy: 0.8996
2025-11-18 22:00:14.730 | INFO | __main__:main:81 - Bagging - Accuracy: 0.8993
2025-11-18 22:01:06.495 | INFO | __main__:main:99 - DecisionTree - Accuracy: 0.9152
```

9. (5%) Plot the feature importance of the decision tree.



(5%) Code Linting

10. Show the snapshot of the flake8 linting result (paste the execution command even when there is no error).

```
(114-1-ml-hw3-release) wuhsinze@Mac 114-1-ml-hw3-release % flake8 main.py
(114-1-ml-hw3-release) wuhsinze@Mac 114-1-ml-hw3-release %
```

Part. 2, Questions (40%):

1. (15%)

In the AdaBoost algorithm, each selected weak classifier, h_t is assigned a weight:

$$\alpha_t = \frac{1}{2} \ln \frac{1-\epsilon_t}{\epsilon_t}$$

(a) What is the definition of ϵ_t in this formula?

(b) If a weak classifier h_t performs only as well as "random guessing" (i.e., $\epsilon_t = 0.5$), what will α_t be?

(c) Based on your answer in (b), briefly explain how this α_t weight formula ensures the robustness of AdaBoost.

(a)

It is the weighted error rate for the t-th weak classifier. It is defined as the sum of the weights of all training samples that the weak classifier misclassified in that specific iteration.

(b)

If $\epsilon_t = 0.5$, then the alpha would become 0

(c)

For the classifier performing random guessing, would be given a zero weight, which means AdaBoost would ignore those random guessing, focusing on hard-to-classify samples, and combines many slightly better-than-random classifiers to build a strong overall model.

2. (15%) Random Forest is often considered an improvement over Bagging (Bootstrap Aggregating) when used with Decision Trees as the base learners.

(a) Briefly explain the potential problem that can exist among the T classifiers

($C_1, \dots, C_T$) produced by Bagging when using decision trees.

(b) To mitigate the problem from (a), Random Forest introduces a second source of randomness in addition to Bagging. Specifically describe what this second source of randomness is and how it works.

(a)

Bagging improves stability and accuracy, but its effectiveness depends on the diversity and quality of the base classifiers and the characteristics of the data. If the training data is not diverse enough, the classifiers may be highly correlated. Decision trees are prone to overfitting, especially if they are grown deep without pruning.

Bagging can reduce variance, but if each tree overfits, the ensemble may still overfit.

(b)

In addition to bagging (random sampling of data), Random Forest randomly selects a subset of features (columns) at each split in the decision tree, rather than considering all features.

When building each tree, at every node (split), only a random subset of features is considered for splitting, not all features. This forces trees to be more diverse, as different trees may use different features to make decisions, even if they see similar data. As a result, the trees are less correlated, and the ensemble benefits more from averaging their predictions.

3. (10%) Is it always possible to find the smallest decision tree that codes a dataset with no error in polynomial time? If not, what algorithm do we usually use to search a decision tree in a reasonable time? Also, list the criterion that we stop splitting the decision tree and leaf nodes are created (List two).

No

Id3, C4.5, CART

1. The node is pure

All samples in that node have the same class, so no further split is needed.

2. No good split is possible

Either no attribute improves impurity/information gain, or no attributes are left. The node becomes a leaf with the majority class.